Tom’s House Out of Order Processor

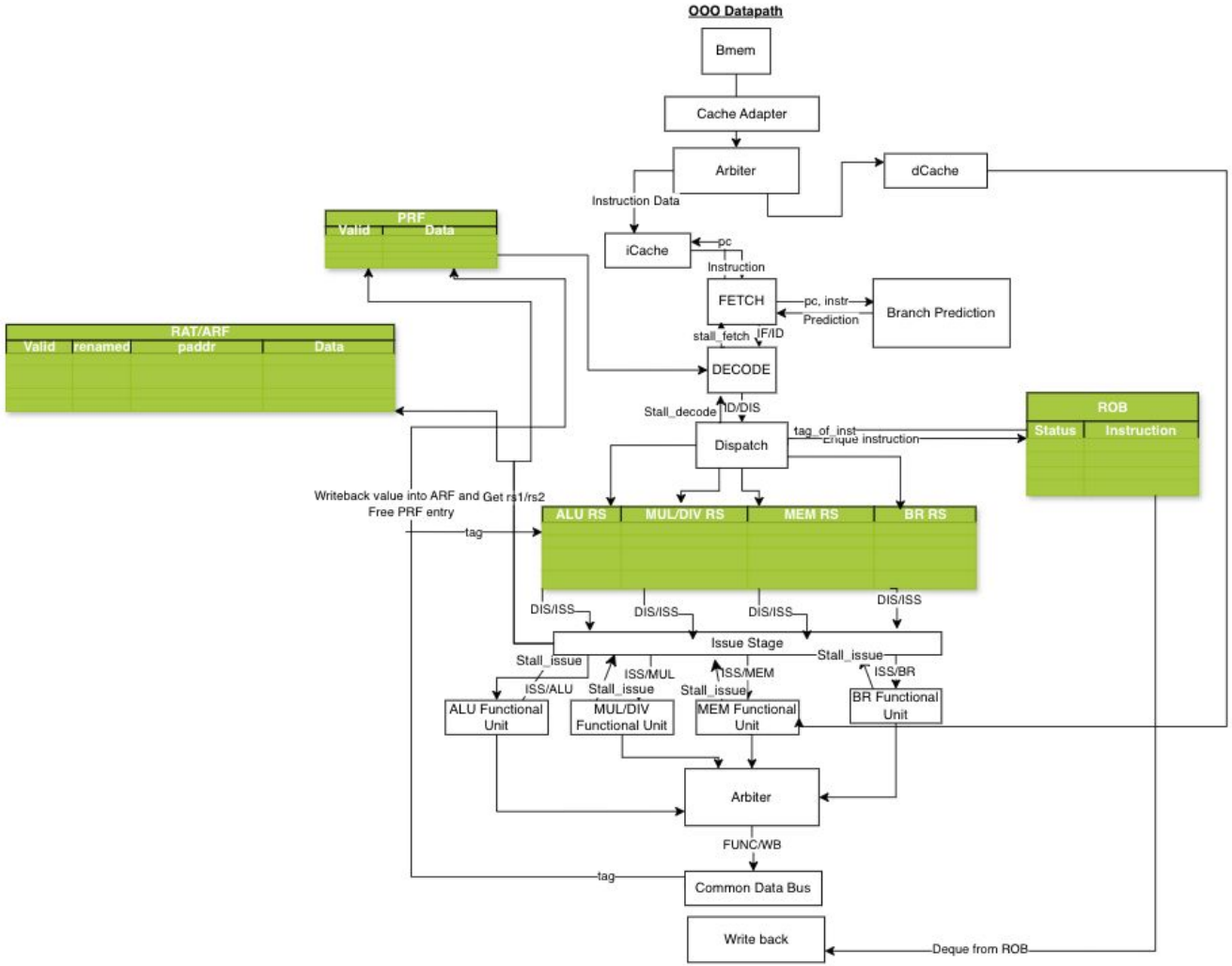
Tom Boston, Hala Mansour, Rohan Nagaraj

Processor Features

- Tomasulo’s Based Processor
  - Despite this, still has qualities of ERR
    - Separate RAT and PRF
- 16 ROB Entries and PRF Entries
- 4 Memory, Multiply/Divide, and Branch Reservation Station Entries
- 8 ALU Reservation Station Entries
- CDB bus goes into ROB and all Reservation Stations for entry wakeup

Processor Features

- Gshare/Gselect Branch predictor with Branch Target Buffer
- Split LSQ
- Dcache and Icache Prefetcher
- Return Address Stack
- Age Ordered Issue


OOO Datapath
Bmem
Cache Adapter
Arbiter
dCache
Instruction Data
PRF
Valid
Data
iCache
pc
Instruction
FETCH
pc, instr
Prediction
Branch Prediction
stall_fetch
IF/ID
RAT/ARF
Valid
renamed
paddr
Data
DECODE
Stall_decode
ID/DIS
ROB
Status
Instruction
Dispatch
tag_of_inst
Enque instruction
Writeback value into ARF and Get rs1/rs2
Free PRF entry
tag
ALU RS
MUL/DIV RS
MEM RS
BR RS
DIS/ISS
DIS/ISS
DIS/ISS
DIS/ISS
Issue Stage
Stall_issue
Stall_issue
ISS/MUL
ISS/MEM
ISS/BR
ISS/ALU
Stall_issue
Stall_issue
ALU Functional Unit
MUL/DIV Functional Unit
MEM Functional Unit
BR Functional Unit
Arbiter
FUNC/WB
tag
Common Data Bus
Write back
Deque from ROB

Branch Predictor

- Branch Target Buffer (BTB) + gshare PHT
- On instruction fetch, the PC indexes a direct-mapped BTB
- If the BTB entry is valid and the tag matches:
  - A 2-bit saturating counter is read from the PHT
  - PHT index computed as: `pht_index_pred = pc[6:2] ^ ghr[4:0]`
  - The counter's MSB predicts taken / not taken:
    - `if (pht[pht_index_pred][1]) pc_next = bht[idx].target`
- Predictor state updated on branch commit
  - BTB target is updated with the resolved address
  - PHT counter is incremented or decremented
  - GHR is shifted in with the actual branch outcome (1 if taken, 0 if not)

Branch Predictor IPC Difference

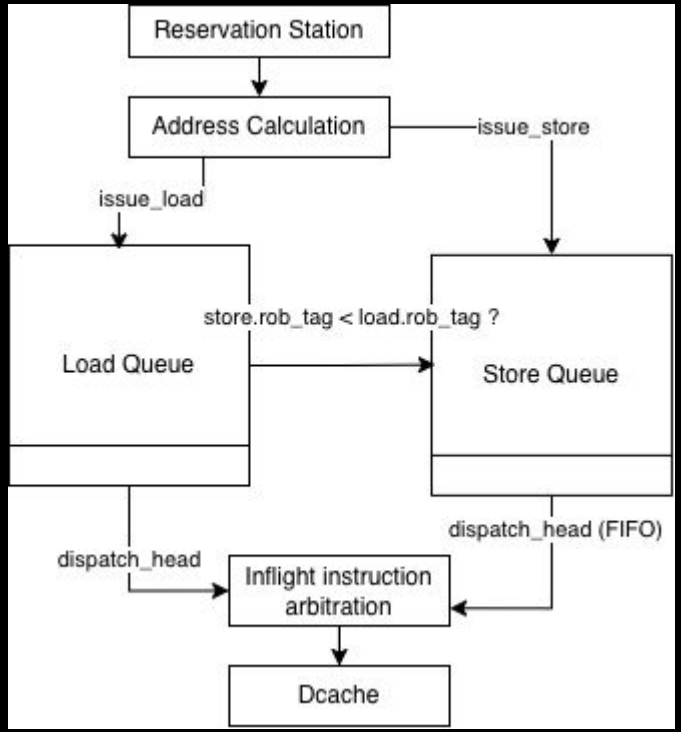
Benchmark	IPC (No Branch Pred)	IPC (With Branch Pred)
Coremark	0.393239	0.5023
AES_SHA	0.465294	0.4691
fft	0.549612	0.59
Compression	0.464641	0.5994
Mergesort	0.452682	0.4859
*clock set to 250 MHz for all tests		

- As we can infer from the data, its effective for tight loops and repetitive control-flow patterns
  - Showcased in Compression, Coremark, and FFT

Split LSQ

1. How it works ?
2. Bugs
3. Attempt at load forwarding

	IPC Recorded Pre-Split LSQ	IPC Recorded Post-Split LSQ
Merge sort	0.4148	0.436554
Aes sha	0.3503	0.391045
FFT	0.4170	0.4848
Compression	0.4745	0.525771
Coremark	0.412468	0.43875



DCache/ICache Prefetcher

- Icache next-line prefetcher
  - Requests sent to memory (alloc state) sends two requests
    - Fetch address and fetch address + 32
  - Generates a “prefetched” line buffer, forwarded to fetch stage and icache adapter
    - Fetch stage uses data on a hit, and gets placed in cache in the meantime
    - Only placed into cache on a hit
- Dcache stride prefetcher
  - Records read request history and determines stride
    - Stride = addr(current) - addr(previous)
    - Prefetch addr(current) + stride
  - Also stored into a line buffer and placed into cache on a hit
  - Not stored in cache on a miss to prevent overloading cache with garbage data
  - Ignores stride if the stride value is really large
    - Very unlikely that this address is used, so just fetch next-line instead

Return Address Stack

- Keeps a speculative and committed version of RAS
  - Committed version is only updated on push/pop on commit
  - Speculative version is updated during fetch
  - Speculative <= committed on pipeline flush
- Push and pop according to RISC-V standard
- If there are unresolved “call” instructions, ignore RAS
  - Might not be the right target address
  - In hindsight, there might be better ways to perform these actions

rd is *x1*/*x5*	*rs1* is *x1*/*x5*	*rd*=*rs1*	RAS action
No	No	—	None
No	Yes	—	Pop
Yes	No	—	Push
Yes	Yes	No	Pop, then push
Yes	Yes	Yes	Push

Age Ordered Issue

- RVFI Order recorded for each in-flight instruction
  - In RS, issue lowest-order ready instruction
  - Results in area and critical path problems
  - Not included in our final revision
- How we would implement this now
  - In hindsight, age-order issue could have been implemented using ROB index
    - We did this in Split LSQ to determine if loads were older than stores
  - Age = instruction ROB head pointer - instruction ROB index
    - Age is the number of instructions that need to commit before this one commits
    - In this case, we can issue the ready instruction with the lowest age

Things to be done better

- More focus on Tomasulo’s or ERR documentation
  - Came up with our own register renaming, RS, ROB, branch flushing implementations
    - Led to inefficiencies and high area
- Less structs, think of area considerations while writing code
- Testing more parameters (Split LSQ too large, RS was uniform)
- Better CDB arbitration logic